

Anforderungsspezifikation & Sprint Backlog

Entwicklung & Evaluation eines Konzepts zur automatisierten Transformation
unstrukturierter Produktdaten mittels großer Sprachmodelle

Fabian Lohoff (Matrikelnummer: 9051803)

4. Juni 2026

Zusammenfassung

Dieses Dokument fasst alle funktionalen, nicht-funktionalen und technischen Anforderungen für den Projektteil der Bachelorarbeit zusammen. Es beinhaltet zudem die formulierten agilen User Stories, die als Grundlage für die Sprintplanung und Implementierung der Softwarearchitektur (gemäß aktuellem Kontextdiagramm) dienen.

Inhaltsverzeichnis

1	Einleitung und Systemkontext	2
2	Funktionale Anforderungen (FA)	3
2.1	Phase 1: Input & Vorverarbeitung	3
2.2	Phase 2: Extraktion & LLM-Steuerung	3
2.3	Phase 3: Normalisierung & Regelbasierter Vergleich	3
2.4	Phase 4: Produkt-Matching & API-Übertragung	3
3	Nicht-Funktionale Anforderungen (NFR)	3
4	Technische Anforderungen (TA) & Rahmenbedingungen (RA)	4
5	Agile User Stories (Sprint Backlog)	5
5.1	Epos: Phase 1 – Input & Vorverarbeitung	5
5.2	Epos: Phase 2 – Extraktion & LLM-Steuerung	5
5.3	Epos: Phase 3 – Normalisierung & Regelbasierter Vergleich	5
5.4	Epos: Phase 4 – Produkt-Matching & API-Übertragung	6
6	Sprintplanung	7
6.1	Sprint 1: Hauptmodul (Transformation & Mapping)	7
6.2	Sprint 2: Normalisierungs- & Vergleichsmodul	7

1 Einleitung und Systemkontext

Das in Abbildung 1 dargestellte Kontextdiagramm veranschaulicht die informationstechnische Einbettung und den Datenfluss des zu entwickelnden Transformations-Systems:

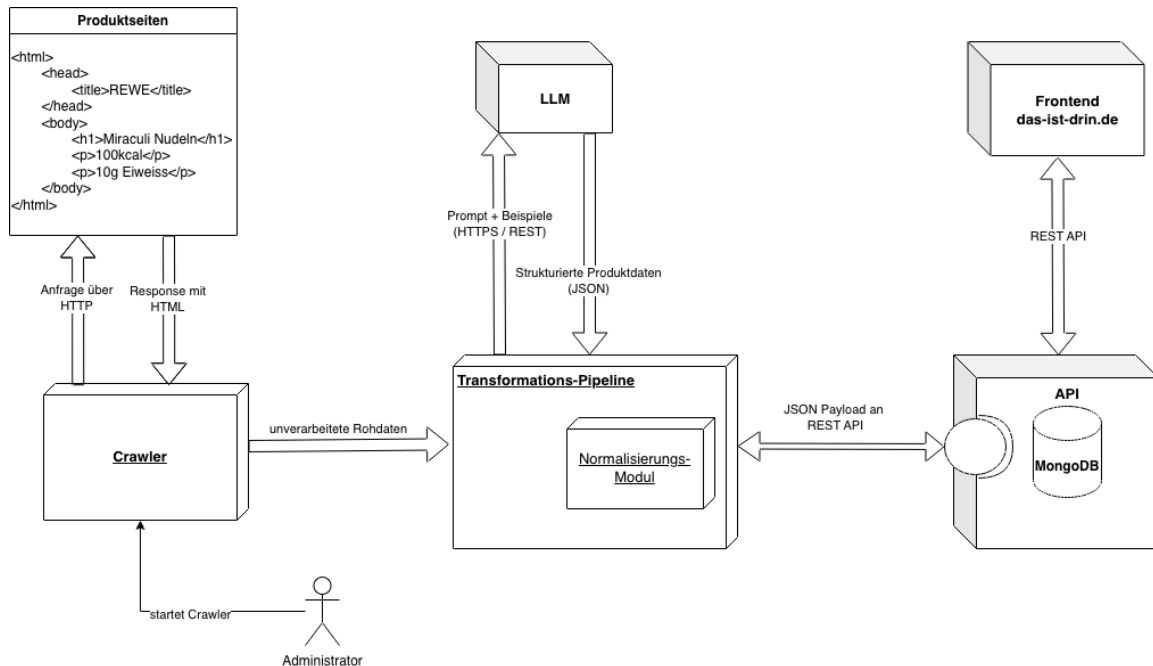


Abbildung 1: Systemkontext der Transformations-Pipeline mit integriertem Normalisierungs-Modul

Der Gesamtprozess gliedert sich in folgende Kernkomponenten:

- 1. Crawler & Vorverarbeitung:** Ein Administrator initiiert den Crawler, welcher Produktseiten von Ziel-Websites (z. B. REWE) lädt und das unstrukturierte HTML-Rohmaterial an die Transformations-Pipeline übergibt.
- 2. LLM-Schnittstelle (Gemini API):** Die in Python implementierte Transformations-Pipeline kommuniziert via HTTPS/REST mit der Gemini API. Über strukturierte Prompts wird das LLM angewiesen, die unstrukturierten Daten semantisch zu interpretieren und als strukturiertes JSON-Objekt zurückzugeben.
- 3. Normalisierungs-Modul (Das Extra-Modul):** Nachgelagert an die LLM-Schnittstelle befindet sich ein isoliertes, regelbasiertes Softwaremodul. Dieses normalisiert alle extrahierten Größen- und Mengenangaben deterministisch in vordefinierten Basiseinheiten (Gramm und Milliliter) und dient gleichzeitig als wissenschaftliche Vergleichs-Baseline.
- 4. KI-basiertes Produkt-Matching & API-Anbindung:** Vor der finalen Datenübergabe erfolgt ein KI-gestützter Dublettenabgleich mit dem Bestandsdatenpool. Schließlich wird das bereinigte JSON-Payload an eine bereitgestellte REST-API übergeben. Diese API fungiert als sicherer Gatekeeper zur Validierung und Persistierung innerhalb der MongoDB, auf die das Frontend von *das-ist-drin.de* zugreift. Die Pipeline selbst besitzt somit keine direkten Datenbanktreiber oder Schreibrechte.

2 Funktionale Anforderungen (FA)

2.1 Phase 1: Input & Vorverarbeitung

- **FA 1.1 (HTML-Cleaning):** Das System muss HTML-Rohtexte von im Kontext irrelevantem Ballast (Scripts, Footer, Navigationsleisten) befreien, um Token-Kosten zu sparen und das „Rauschen“ für das LLM zu minimieren.
- **FA 1.2 (Batching):** Das System muss in der Lage sein, Datensätze sequenziell oder in konfigurierbaren Batches zu verarbeiten, um API-Rate-Limits nicht zu verletzen.

2.2 Phase 2: Extraktion & LLM-Steuerung

- **FA 2.1 (JSON-Contract):** Das System muss das LLM (Gemini API) über strukturierte Prompts (z. B. via Structured Outputs / JSON-Mode) dazu zwingen, ausschließlich ein valides, vordefiniertes JSON-Format zurückzugeben.
- **FA 2.2 (Hierarchische Extraktion):** Das System muss komplexe, geschachtelte und unstrukturierte Produktstrukturen aus den Webdaten auflösen und in flache oder sauber strukturierte Objekte überführen.

2.3 Phase 3: Normalisierung & Regelbasierter Vergleich

- **FA 3.1 (Einheiten-Normalisierung):** Nach der Überführung in strukturierte Daten müssen alle extrahierten Größen- und Mengenangaben (z. B. KG, L, g, ml) in standardisierte Basiseinheiten transformiert werden. Jedes Gewicht muss in Gramm (g) und jede Flüssigkeitsmenge in Milliliter (ml) normalisiert werden (z. B. Typ-Konvertierung von „Fett: 12g“ in `fats.value: 12` und `fats.unit: "g"`).
- **FA 3.2 (Regelbasierte Extraktion & Evaluation):** In diesem Modul muss zusätzlich ein rein regelbasierter Ansatz (z. B. über reguläre Ausdrücke/Heuristiken analog zu klassischen Wrapper-Verfahren oder Named-Entity-Recognition-Ansätzen wie FoodIE) zur Extraktion und Normalisierung von Verpackungsgrößen implementiert werden. Dieser dient als deterministischer Vergleichsmaßstab, um die Güte der LLM-Extraktion zu evaluieren.

2.4 Phase 4: Produkt-Matching & API-Übertragung

- **FA 4.1 (KI-basiertes Produkt-Matching):** Nach der Transformation und Normalisierung müssen die Produktdaten mit dem bereits existierenden Datenbestand abgeglichen werden (z. B. Prüfung: „Existiert diese Cola-Flasche bereits in der DB?“). Dies soll KI-gestützt (z. B. mittels Few-Shot-Prompting) geschehen, um redundante Datensätze zu vermeiden.
- **FA 4.2 (API-Datenübergabe):** Das bereinigte, normalisierte und abgeglichene Produktobjekt darf nicht direkt in die Datenbank geschrieben werden. Das System muss die strukturierten Daten stattdessen an eine bereitgestellte API übergeben, welche die finale Validierung und Persistierung übernimmt.

3 Nicht-Funktionale Anforderungen (NFR)

- **NFR 1 (Robustheit / Schema-Compliance):** Mindestens 95 % der vom System aufbereiteten JSON-Objekte müssen ohne strukturellen Fehler direkt vom Schema der Ziel-API akzeptiert werden.

- **NFR 2 (Kostenkontrolle & Wirtschaftlichkeit):** Die durchschnittlichen API-Kosten der Gemini-Aufrufe pro transformierter Produktseite dürfen einen Betrag von X Cent (z. B. 1–2 Cent) nicht überschreiten.

4 Technische Anforderungen (TA) & Rahmenbedingungen (RA)

- **TA 1 (Programmiersprache):** Das gesamte System sowie das Normalisierungs-Modul müssen vollständig in Python implementiert werden.
- **TA 2 (Schnittstellen-Architektur):** Die Datenübergabe erfolgt ausschließlich über eine vorgegebene REST- oder GraphQL-API. Das zu entwickelnde Modul besitzt keine direkten Schreibrechte oder direkten Treiberverbindungen (wie z. B. native `pymongo`-Verbindungen) zur produktiven MongoDB.
- **RA 1 (LLM-Infrastruktur):** Als Large Language Model zur Extraktion und zum Produkt-Matching muss die Gemini API verwendet werden.

5 Agile User Stories (Sprint Backlog)

Die nachfolgenden User Stories sind nach den INVEST-Merkmalen formuliert und folgen der Standardstruktur nach Mike Cohn.

5.1 Epos: Phase 1 – Input & Vorverarbeitung

US 1.1: HTML-Cleaning zur Rausch- und Kostenreduktion

Als Daten-Engineer **möchte ich** unstrukturierte HTML-Rohtexte automatisch von irrelevanten Elementen befreien, **um** das semantische Rauschen für das LLM zu minimieren und Token-Kosten zu sparen.

Akzeptanzkriterien:

- Ein Python-Skript entfernt alle `<script>`, `<style>`, `<footer>` und `<nav>`-Tags.
- Der verbleibende Text enthält primär Produktinformationen.
- Die Zeichenanzahl wird im Vergleich zum Roh-HTML um mindestens 40 % reduziert.

US 1.2: Konfigurierbare Batch-Verarbeitung gegen Rate-Limits

Als System-Administrator **möchte ich** Datensätze sequenziell oder in konfigurierbaren Batches verarbeiten lassen können, **um** die API-Rate-Limits der Gemini API nicht zu verletzen.

Akzeptanzkriterien:

- Batch-Größe und Delay sind über eine Konfigurationsdatei (z. B. `config.yaml`) steuerbar.
- System fängt 429 (Too Many Requests)-Fehlermeldungen ab und reagiert mit Exponential Backoff.

5.2 Epos: Phase 2 – Extraktion & LLM-Steuerung

US 2.1: Strukturierte und hierarchische LLM-Datenextraktion

Als Daten-Analyst **möchte ich** die Gemini API über strukturierte Prompts so steuern, dass geschachtelte Produktstrukturen aufgelöst und zwingend in einem vordefinierten JSON-Format zurückgegeben werden.

Akzeptanzkriterien:

- Die Extraktion nutzt das Feature für *Structured Outputs* (JSON-Mode).
- Das JSON entspricht exakt den Vorgaben.
- Kosten für API-Aufrufe liegen stabil bei unter 1–2 Cent pro Seite (Erfüllung NFR 2).

5.3 Epos: Phase 3 – Normalisierung & Regelbasierter Vergleich

US 3.1: Einheiten-Normalisierung für Mengenangaben

Als Backend-Entwickler **möchte ich** ein isoliertes Normalisierungs-Modul in Python implementieren, das alle extrahierten Größen in standardisierte Basiseinheiten (g und ml) konvertiert.

Akzeptanzkriterien:

- Strings wie "0.5 kg" werden deterministisch in numerische Werte und Einheiten gesplittet.
- Gewichte werden mathematisch korrekt in g und Flüssigkeiten in ml umgerechnet.
- Modul agiert als eigenständige Pipeline-Komponente ohne DB-Anbindung.

US 3.2: Regelbasierte Verpackungsgrößen-Extraktion als Baseline

Als Bachelorand **möchte ich** einen rein regelbasierten Ansatz zur Extraktion von Verpackungsgrößen entwickeln, **um** eine verlässliche Baseline für den wissenschaftlichen Vergleich mit dem LLM zu haben.

Akzeptanzkriterien:

- Extraktion erfolgt über Regex oder Heuristiken (analog *FoodIE*).
- Das Modul gibt die Ergebnisse getrennt aus, sodass ein direkter Vergleich von Precision, Recall und F1-Score möglich ist.

5.4 Epos: Phase 4 – Produkt-Matching & API-Übertragung

US 4.1: KI-gestütztes Produkt-Matching zur Duplikatsvermeidung

Als Daten-Engineer **möchte ich**, dass das System transformierte Produktdaten mittels Gemini API gegen den Bestand abgleicht, **um** redundante Datensätze zu verhindern.

Akzeptanzkriterien:

- Few-Shot-Prompting wird zur semantischen Ähnlichkeitsbewertung genutzt.
- Bestandsdaten für den Abgleich werden dynamisch über die Ziel-API abgefragt.

US 4.2: Entkoppelte Datenübergabe über die REST-API

Als Software-Architekt **möchte ich**, dass das fertige Produktobjekt ausschließlich an eine API übergeben wird, **damit** das System von der MongoDB entkoppelt bleibt.

Akzeptanzkriterien:

- Keine direkten Datenbank-Treiber (*pymongo*) innerhalb der Pipeline vorhanden.
- Die Pipeline kommuniziert ausschließlich per HTTP-Post mit der Ziel-API.
- Mindestens 95 % der JSON-Objekte werden vom Schema der API akzeptiert (Erfüllung NFR 1).

6 Sprintplanung

Die informationstechnische Umsetzung des Gesamtsystems wird in zwei diskrete Sprints unterteilt, um eine inkrementelle und wissenschaftlich evaluierbare Entwicklung zu gewährleisten.

6.1 Sprint 1: Hauptmodul (Transformation & Mapping)

Zeitraum: 15. Juni bis 05. Juli (3 Wochen)

Fokus: Kernarchitektur der Pipeline, Datenvorverarbeitung, die eigentliche LLM-basierte Extraktion sowie das nachgelagerte Matching und die Bereitstellung der API-Übergabe.

- **Zugeordnete User Stories:**

- ◇ **US 1.1:** HTML-Cleaning zur Rausch- und Kostenreduktion
- ◇ **US 1.2:** Konfigurierbare Batch-Verarbeitung gegen Rate-Limits
- ◇ **US 2.1:** Strukturierte und hierarchische LLM-Datenextraktion
- ◇ **US 4.1:** KI-gestütztes Produkt-Matching zur Duplikatsvermeidung
- ◇ **US 4.2:** Entkoppelte Datenübergabe über die REST-API

- **Sprint-Ziel:** Am Ende von Sprint 1 können unstrukturierte HTML-Seiten eingelesen, bereinigt, durch das LLM (Gemini) in valide JSON-Strukturen transformiert, mit Altdaten gematcht und fehlerfrei an die bereitgestellte REST-API übergeben werden.

6.2 Sprint 2: Normalisierungs- & Vergleichsmodul

Zeitraum: 06. Juli bis 19. Juli (2 Wochen)

Fokus: Entwicklung des isolierten Moduls zur Standardisierung der Mengenangaben sowie Implementierung des deterministischen, regelbasierten Alternativansatzes (Baseline) zur Fertigstellung aller Programmierarbeiten.

- **Zugeordnete User Stories:**

- ◇ **US 3.1:** Einheiten-Normalisierung für Mengenangaben
- ◇ **US 3.2:** Regelbasierte Verpackungsgrößen-Extraktion als Baseline

- **Sprint-Ziel:** Integration des mathematischen Normalisierungsschritts (Umrechnung in `g` und `ml`) als isolierte Pipeline-Komponente und Bereitstellung des regelbasierten Wrapper-Verfahrens zur quantitativen Evaluierung innerhalb der empirischen Analyse. Damit sind sämtliche Software-Artefakte für die Thesis abgeschlossen.